

DEVOPS LAB-4

CI/CD PIPELINE USING JENKINS, GRADLE AND APACHE TOMCAT

Practical Report

Student: Vijay

Date: 17 August 2026

Platform: Windows + Jenkins + GitHub + Gradle + Apache Tomcat 11

Objective: Build a Jenkins pipeline that checks out source code, builds/tests the application, packages the deployable artifact, and deploys it to Tomcat.

1. Aim

To build a CI/CD pipeline in Jenkins using a Java project hosted on GitHub, use Gradle as the build tool, package the web application for deployment, and deploy the application to Apache Tomcat.

2. Lab Requirement

The reference laboratory specifies a pipeline flow of GitHub → Jenkins Pipeline → Checkout → Build → Test → Package → WAR → Deploy → Apache Tomcat → browser verification. It also describes a first level using Maven + Tomcat and a second level replacing Maven with Gradle or Ant. [filecite?url=urn5file0%2FL11-L38%2F](#)

For the demonstrated implementation, Jenkins was already running on port 8080, so Tomcat was configured on port 8081 to avoid a port conflict. The reference explicitly recommends Jenkins on 8080 and Tomcat on 8081.

[filecite?url=urn5file0%2FL58-L70%2F](#)

3. Software / Tools Used

- Jenkins – CI/CD automation server
- Git and GitHub – source-code management
- Java JDK 21 – Java compilation/runtime
- Gradle – build automation
- Apache Tomcat 11.0.24 – application server
- Google Chrome – application verification

4. System Configuration

Jenkins: <http://localhost:8080>

Tomcat: <http://localhost:8081>

GitHub repository: <https://github.com/vijayyyh/ci-jenkins-multi-build-demo>

Tomcat installation directory observed: C:\Program Files\Apache Software Foundation\Tomcat 11.0

5. Apache Tomcat Configuration

Apache Tomcat 11.0.24 was installed and configured to use HTTP port 8081. The Tomcat home page was successfully opened at localhost:8081, confirming that the server itself was installed and running.

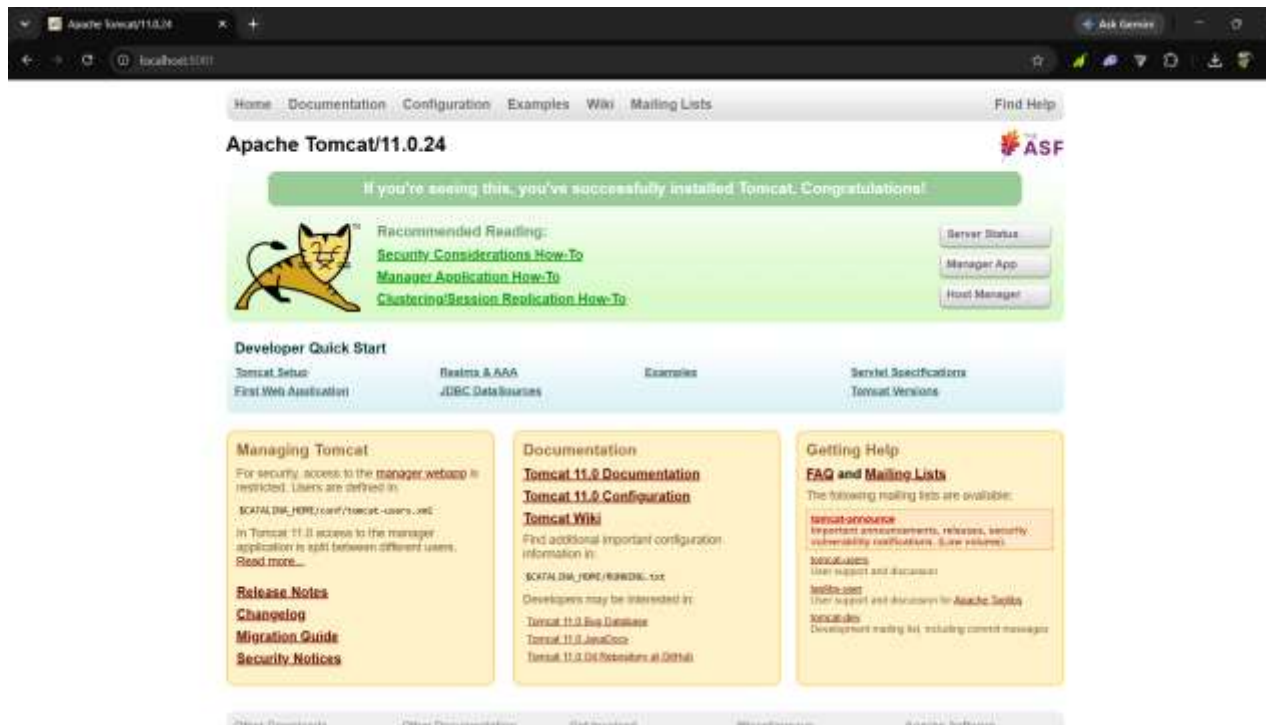


Figure 1. Apache Tomcat 11.0.24 running successfully on localhost:8081.

6. Source Code Repository

The application source was maintained in the public GitHub repository ci-jenkins-multi-build-demo. The repository contains the Java source/test directories together with build.gradle, build.xml, pom.xml and settings.gradle.

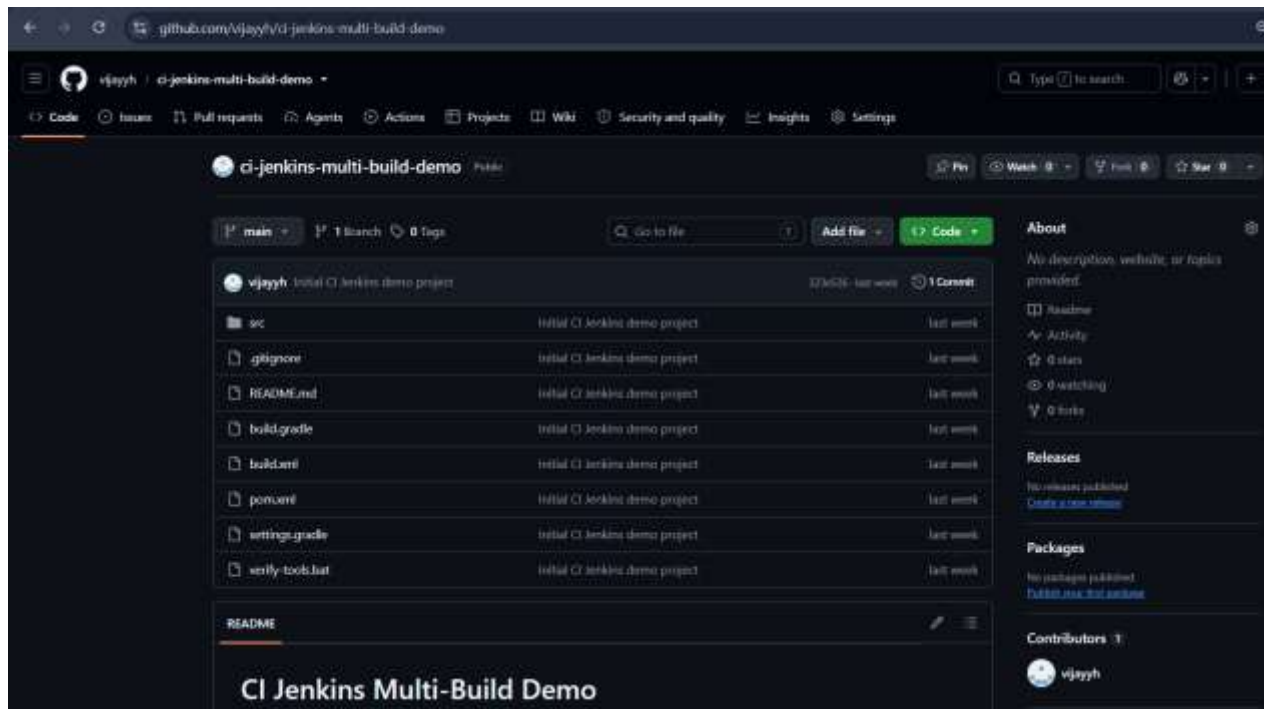


Figure 2. GitHub repository containing the CI Jenkins multi-build demo project.

7. Web Application and Gradle Build

The project was adapted for the Tomcat demonstration by adding a web application under `src/main/webapp` and modifying `build.gradle` to provide the required web packaging/deployment configuration. The Gradle project uses Java toolchain version 21 and includes a `demoTest` task used during the build.

The earlier Gradle CI job was successfully completed before the Tomcat pipeline work; its Gradle build output reported `BUILD SUCCESSFUL` and the demo test passed.

8. Jenkins Pipeline Configuration

A Jenkins Pipeline job named CI-Tomcat-Pipeline was created. The pipeline configuration contains three intended stages: Checkout, Build and Deploy to Tomcat. The Checkout stage retrieves the main branch from GitHub; the Build stage invokes Gradle; and the Deploy stage copies the generated WAR file into the Tomcat webapps directory.

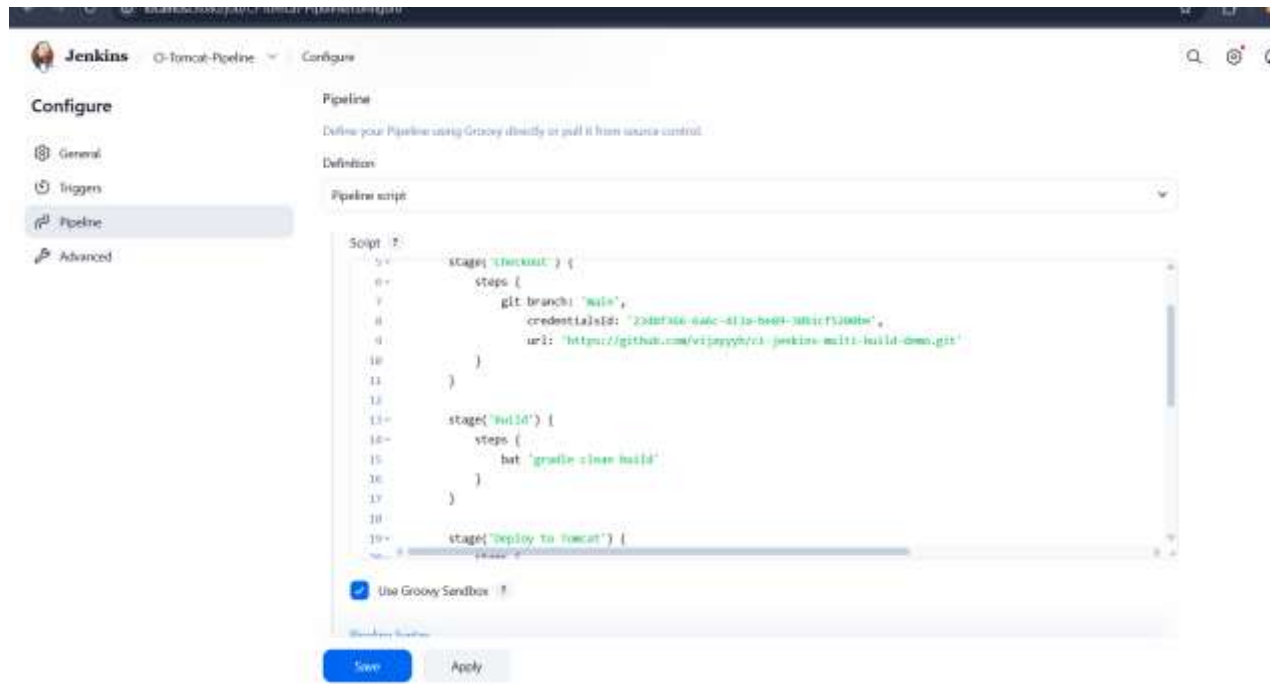


Figure 3. Jenkins CI-Tomcat-Pipeline configuration with Checkout, Build and Deploy stages.

Pipeline Script Used

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main',
                    credentialsId: '23d8f366-6a6c-413a-be89-38b1cf5200be',
                    url: 'https://github.com/vijayyyh/ci-jenkins-multi-build-demo.git'
            }
        }

        stage('Build') {
            steps {
                bat 'gradle clean build'
            }
        }

        stage('Deploy to Tomcat') {
            steps {
                bat '''
                    copy /Y "%WORKSPACE%\build\libs\ci-jenkins-demo-1.0.0.war" ^
                    "C:\Program Files\Apache Software Foundation\Tomcat 11.0\webapps\ci-jenkins-demo.war"
                '''
            }
        }
    }
}
```

```

    }
}

post {
    success {
        echo 'Build, test and deployment completed successfully!'
    }
    failure {
        echo 'Pipeline failed.'
    }
}
}
}

```

9. Pipeline Execution Evidence

Several manual builds were triggered while configuring and troubleshooting the pipeline. Jenkins reported SUCCESS for the builds shown below; however, the console output captured during these attempts only displayed the Declarative Post Actions block rather than the expected Checkout, Build and Deploy stages.

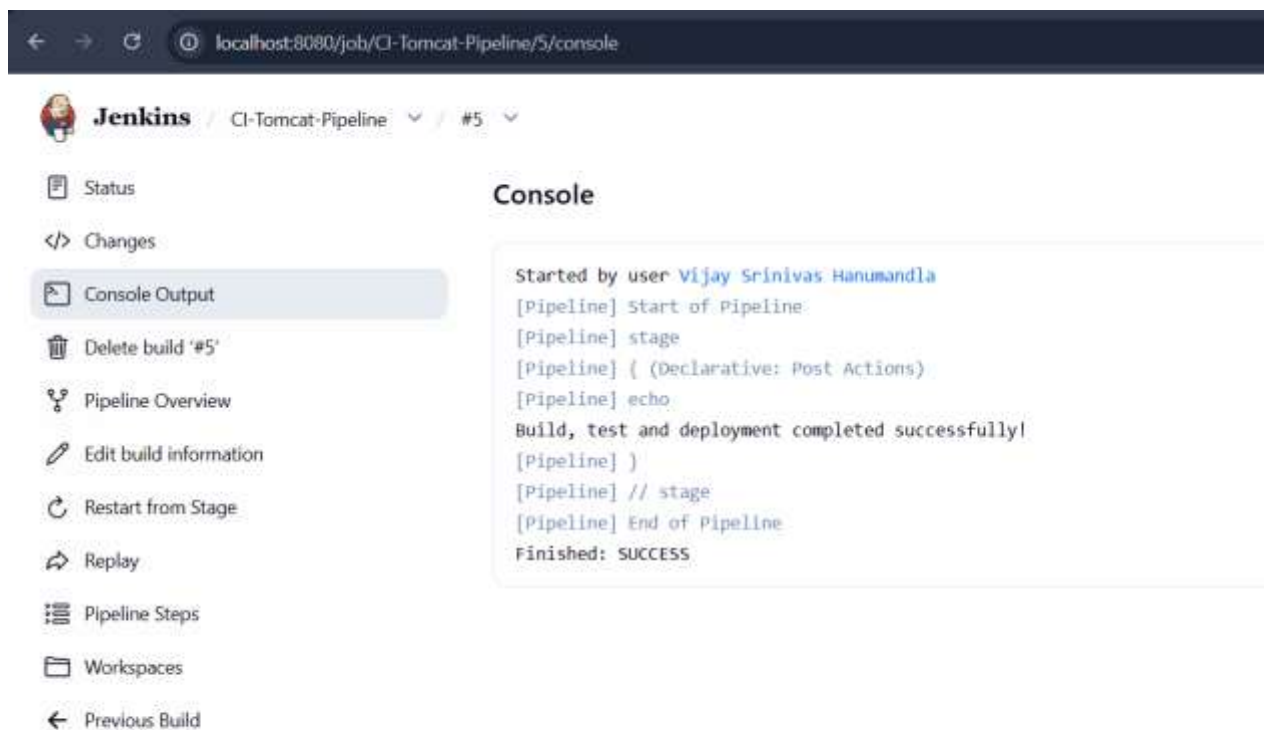


Figure 6. Jenkins CI-Tomcat-Pipeline Build #5 console output.

Status

Changes

Console Output

Delete build '#3'

Pipeline Overview

Edit build information

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

Console

```
Started by user Vijay Srinivas Hanumandla
[Pipeline] Start of Pipeline
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Build, test and deployment completed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 7. Jenkins CI-Tomcat-Pipeline Build #4 result page.

10. Deployment Verification

Tomcat itself was verified successfully at localhost:8081. The application context /ci-jenkins-demo/ was then tested. The captured result was HTTP 404, meaning Tomcat was running but the application had not yet been deployed at that context.



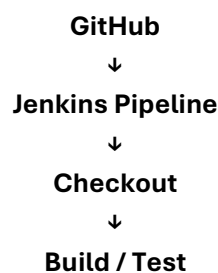
Figure 4. Initial Tomcat deployment verification showing HTTP 404 for the application context.

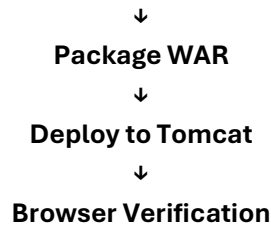
This is an important distinction: a green Jenkins build message by itself does not prove that the WAR reached Tomcat. The reference lab requires both deployment and browser verification. [filecite?turn5file2?L322-L396?](#)

11. Troubleshooting Performed

- Confirmed Jenkins is available on port 8080.
- Confirmed Tomcat is available on port 8081.
- Confirmed the GitHub repository is accessible and contains the project files.
- Created and configured the CI-Tomcat-Pipeline Jenkins job.
- Added Checkout, Build and Deploy stages to the Pipeline script.
- Triggered multiple Jenkins builds and inspected Console Output.
- Tested the deployed application URL and identified the HTTP 404 deployment issue.

12. Reference Pipeline Flow





The laboratory reference explicitly defines Checkout, Compile, Test, Package, Deploy and Verify as the major pipeline activities. [filecite2turn5file2L289-L396](#)

13. Result / Current Status

Tomcat installation and port configuration were completed successfully. The GitHub repository and Jenkins Pipeline job were also configured. The remaining practical issue is the final deployment execution: the current Jenkins builds are reporting SUCCESS while not showing the expected Checkout/Build/Deploy stages in Console Output, and the application URL currently returns HTTP 404.

Therefore, the environment and pipeline configuration are documented, but the final browser-level deployment verification should be completed before claiming the experiment as fully successful.

14. Conclusion

This experiment demonstrated the configuration of a Jenkins-based CI/CD workflow integrating GitHub, Gradle and Apache Tomcat. The practical established the required server ports, source repository, Jenkins Pipeline configuration and WAR deployment approach. Final submission should include the successful Jenkins stage execution and the deployed application page once the current pipeline execution issue is resolved.

15. Screenshot Index

1. Figure 1. Apache Tomcat 11.0.24 running successfully on localhost:8081.
2. Figure 2. GitHub repository containing the CI Jenkins multi-build demo project.
3. Figure 3. Jenkins CI-Tomcat-Pipeline configuration with Checkout, Build and Deploy stages.
4. Figure 4. Initial Tomcat deployment verification showing HTTP 404 for the application context.
6. Figure 6. Jenkins CI-Tomcat-Pipeline Build #5 console output.
7. Figure 7. Jenkins CI-Tomcat-Pipeline Build #4 result page.
8. Figure 8. Jenkins CI-Tomcat-Pipeline general configuration page.

Report prepared from the DevOps Lab-4 reference material and the screenshots captured during the practical session.